Technische Universität München

Übungen Rechnereinführung WS 2026/27

Adrian Stein – Thomas Lindae – Ari Breitkreuz

---

2 – Linux (Teil 2)

Aufgaben, welche mit einem (*) markiert sind, sind Bonusaufgaben und sollten nur bearbeitet werden, wenn man die Zeit dafür hat.

2.1 Package Management

In diesem Teil geht es darum, den Umgang mit Package Managern ein bisschen zu üben. Deswegen werden wir 4 kleine Kommandos installieren. Überprüfen Sie stets bevor Sie ein Kommando installieren, ob dieses nicht bereits installiert ist. Versuchen Sie es einfach einmal auszuführen. Damit erfahren Sie zum einen, ob das Kommando bereits installiert ist, zum anderen werden Sie, falls das Kommando nicht installiert ist, ggf. von der resultierenden Fehlermeldung eine kleine Anleitung bekommen wie das Kommando mit Ihrem Package Manager zu installieren ist.

2.1.1

Installieren Sie bitte die folgenden Kommandos:

1. `fortune`
2. `cowsay`
3. `sl`
4. `lolcat`

Hinweis: Nutzer von

- *Fedora* benutzen den vorinstallierten Package Manager `dnf`.
- *Ubuntu* benutzen den vorinstallierten Package Manager `apt`.
- *Windows* benutzen das in der ersten Übung aufgesetzte *WSL*. Unter *WSL* ist *Ubuntu* der Standard. Sie benutzen also auch den Package Manager `apt` (Sie müssen u. U. zuerst `sudo apt update` ausführen bevor Sie packages installieren können).

- *MacOS* benutzen Homebrew (`brew`). Diesen Package Manager sollten Sie bereits während der ersten Übung installiert haben. Eine Hilfestellung für die installation einzelner Kommandos finden Sie wenn sie im Internet nach z. B. „Homebrew fortune“ suchen.

Lösung

Fedora:

- `sudo dnf install fortune-mod`
- `sudo dnf install cowsay`
- `sudo dnf install sl`
- `sudo dnf install lolcat`

Ubuntu bzw. *Windows* (*WSL*):

- `sudo apt install fortune-mod`
- `sudo apt install cowsay`
- `sudo apt install sl`
- `sudo apt install lolcat`

MacOS:

- `brew install fortune`
- `brew install cowsay`
- `brew install sl`
- `brew install lolcat`

2.2 Spaß mit Programmen

1. Führen Sie `lolcat` ohne Argumente aus. Was macht dieses Programm?
   (*Hinweis: Mit Strg+C kommt man aus fast allem raus*)
2. Führen Sie `sl` ohne Kommandos aus. Welchen Tippfehler soll es einem abgewöhnen?
3. Erstellen Sie mit einer geeigneten Pipe einen Regenbogenzug!

4. Erstellen Sie eine Regenbogenkuh, welche Ihnen Weisheiten sagt!
5. Können Sie auch den Text Editor *nano* in Regenbogenfarben benutzen?
6. (*) Benennen Sie eine Datei `a.txt` in `b.txt` um. Sollte dies scheitern (z. B. weil `a.txt` nicht existiert), soll eine leere Datei `b.txt` erstellt werden.
7. (*) Leiten Sie eine beliebige Ausgabe in die Datei `/dev/null`. Geben Sie den Inhalt von `/dev/null` aus. Was fällt Ihnen auf? Nutzen Sie ihr neues Wissen um die Ausgaben des in 6 erstellten Kommandos im Fehlerfall zu reduzieren (Sie müssen sich hierführ u. U. im Internet informieren).

Lösung

1. Regenbogenfarbverlauf für jede Eingabe.
2. Es bestraft einen, wenn man versehentlich `sl`, anstatt `ls` eingibt.
3. `sl | lolcat`
4. `fortune | cowsay | lolcat`
5. `nano | lolcat`
6. `mv a.text b.txt || touch b.txt`
7. `/dev/null` ist ein Loch, in dem alles verschwindet.
   `mv a.txt b.txt 2> /dev/null || touch b.txt`

2.3 Access Control

2.3.1 `chmod`

Überfliegen sie zunächst den `man` Eintrag zu `chmod` und probieren Sie die Rechtevergabe mit diesem online tool aus. Wie hängen die Rechte die vergeben werden mit dem Oktal System zusammen?

Beantworten Sie anschließend folgende Fragen:

1. Geben Sie die Zahlen (von 0 bis 7) an, welche den folgenden Rechten entsprechen:
   a) Schreiben und ausführen
   b) Ausführen und lesen
   c) rw
   d) x

e) Lesen

f) Nichts

2. Geben Sie ein `chmod` Kommando an, mit dem Sie sich selbst die Schreib- und Leserechte auf eine Datei `dummy.txt` wegnehmen. Die Ausführrechte sollen dabei nicht verändert werden.

3. Geben Sie ein `chmod` Kommando an, mit welchem nur die Gruppe alle Rechte an einer Datei `file` erhält. Jeder andere (auch der Owner) soll keine Rechte erhalten.

Lösung

Da wir pro Nutzer/Gruppe/Andere immer drei Arten von „binären“ Rechten (*read*, *write*, *execute*) vergeben wollen, bietet sich die Darstellung im Oktalsystem an (da $2^3 = 8$ und wir so alle drei Rechte in genau einer Stelle kodieren können).

1. a) 3
   b) 5
   c) 6
   d) 1
   e) 4
   f) 0

2. `chmod u-rw dummy.txt`

3. `chmod 070 file`

2.4 SSH

In diesem Teil verbinden wir uns mit der Rechnerhalle und richten ein paar Sachen ein.

2.4.1 Verbindung aufbauen

Benutzen Sie das Kommando `ssh <rechnerkennung>@lxhalle.in.tum.de`, um sich mit der Rechnerhalle zu verbinden (Sie müssen wahrscheinlich die Fingerprints der Rechnerhalle verifizieren).

2.4.2 SSH-Keys

Um sich leichter über SSH zu verbinden, werden sehr oft sogenannte SSH-Keys verwendet. Diese ermöglichen, sich schnell und sicher zu authentifizieren.

In kurz funktioniert es so, dass man einen „Private-Key“ und einen „Public-Key“ hat. Wie die Namen schon verraten, sollte der Private-Key privat sein und *niemals* auf eine andere Maschine gebracht werden. Der Public-Key hingegen wird an andere Maschinen gegeben und ist nur mit dem zugehörigen Private-Key (den nur eine Maschine hat) nützlich.

Die Befehle, welche hier eingegeben werden sind zum Teil sehr lang. Denken Sie daran, dass Sie mit der *Tab* Taste eine automatische Vervollständigung hervorrufen können und ggf. mit *Strg-C* das Kommando abbrechen können.

- Generieren Sie sich auf Ihrer **lokalen Maschine** (entweder aus der SSH-Session ausloggen, oder ein neues Terminal starten) ein Schlüsselpaar mit dem Kommando `ssh-keygen -t ed25519`. Die Dateinamen können alle mit *Enter* ohne Weiteres bestätigt werden (es sei denn, man will etwas ändern).
- Nun muss der Public Key in die Rechnerhalle übertragen werden. Der leichteste Weg dies zu tun ist mit dem Kommando `ssh-copy-id -i ~/.ssh/id_ed25519 <rechnerkennung>@lxhalle.in.tum.de`.[1]
- Sie sollten nun aufgefordert werden, Ihr Passwort von der Rechnerhalle einzugeben. Danach muss ggf. noch „yes“ eingetippt werden, um die Verbindung zu bestätigen.
- Es sollte jetzt möglich sein ohne Verwendung des Passworts der Rechnerkennung sich mit der Rechnerhalle zu verbinden (also mit dem Kommando `ssh <rechnerkennung>@lxhalle.in.tum.de`).

2.4.3 Passwort ändern

Ändern Sie Ihre ITO Credentials über Ucentral (**Achtung**: Die Umstellung kann einen Tag dauern, u. U. sind Sie für die Bearbeitungsdauer von einem Passwort-Login ausgeschlossen. Der Zugang zur Rechnerhalle mit SSH wird so oder so weiterhin möglich sein). Alternativ können Sie auch direkt auf der Rechnerhalle das Command `passwd` benutzen. Benutzen Sie ein sicheres Passwort, welches Sie sich merken können (oder noch besser, verwenden Sie einen Passwortmanager wie z. B. Bitwarden oder KeePassXC). Das Passwort zurückzusetzen würde aufwändig werden…

[1] In manchen Fällen gibt es diesen bequemen Weg leider nicht und man muss den Public Key explizit kopieren und auf dem Server in die Datei `~/.ssh/authorized_keys` einfügen.

2.5 Text Editing im Terminal

In dieser Aufgabe sollen Sie die Terminal Editoren *nano*[2] und *vim*[3] ausprobieren. Die Grundlagen der Bedienung wurde bereits in der Vorlesung angesprochen.

Sie werden nun das *Super-Cat* Shell-Script der Vorlesung abschreiben (einmal mit *nano* und einmal mit *vim*). Natürlich ohne Copy-Paste!

Erstellen Sie zunächste eine leere Datei namens `supercat.sh` mit `touch` (machen Sie die Datei zusätzlich noch ausführbar mit `chmod`). Wenn Sie dann ihren Editor wechseln können Sie einfach die alte Datei löschen oder eine neue erstellen.

```
#!/bin/sh

echo "supercat!"
read -p "enter file globbing pattern: " glob

for file in *"$glob"; do
    if [ -f "$file" ]; then
        echo "> Content of ${file}:"
        cat "$file"
    fi
done
```

Anschließend können Sie die Datei im Terminal mit `./supercat.sh` ausführen.

Wenn Sie sich mehr für *vim* (oder besser: Neovim) interessieren, haben wir folgende Ressourcen die Sie sich mal anschauen können:

- Mastering the Vim Language
- Vim As Your Editor—Introduction
- 0 to LSP: Neovim RC From Scratch

2.6 Shell Scripting

1. Schreiben Sie ein Script für die **sh** Shell, welches alle Dateien (im *CWD*) die in `.txt` enden sammelt, und für jede Datei ihren Namen und die Anzahl der enthaltenen Zeilen ausgibt (Format Beispiel (pro Datei): `foo.txt has 8 lines`)

   Hinweis: Schauen Sie sich an wie der output von `wc` sich ändert, wenn Sie die Datei als CLI Arg übergeben bzw. den Inhalt an *stdin* schicken!

[2] nano Bedienungsanleitung
[3] Falls Sie hilfe brauchen können Sie im Terminal einmal `vimtutor` ausführen.

2. (*) Schreiben Sie ein Script für die **sh** Shell, welches die Zeilenzahl aller im *CWD* enthaltenen Dateien die in `.txt` enden summiert[4] und die Summe als Ergebnis ausgibt (Format Beispiel: `Total lines in all .txt files: 9000`).

Lösung

1.
```sh
#!/bin/sh

for file in *.txt; do
    if [ -f "$file" ]; then
        lines=$(wc -l < "$file")
        echo "$file has $lines lines"
    fi
done
```

2.
```sh
#!/bin/sh

total=0

for file in *.txt; do
    if [ -f "$file" ]; then
        lines=$(cat "$file" | wc -l)
        total=$((total + lines))
    fi
done

echo "Total lines in all .txt files: $total"
```

2.7 Umgang mit großen Dateien/Ausgaben

2.7.1 Goethe's Faust

Für diese Aufgabe finden Sie auf Moodle eine Textdatei (`faust.txt`) von Goethe's „Faust. Der Tragödie erster Teil".

1. Versuchen Sie die Datei mit `cat` auf der Konsole auszugeben. Was können Sie feststellen?
2. Um der Limitation von `cat` entgegen zu wirken, öffnen Sie die Datei einmal mit `more` und einmal mit `less`.
3. Nutzen sie `grep` um zu ermitteln wie oft das (Teil-) Wort „Mephisto" in der Datei vorkommt.

[4] Suchen Sie im Internet nach Möglichkeiten zur Addition zweier Shell-Variablen

4. Nutzen sie `grep` um zu ermitteln wie oft das (Teil-) Wort „Gretchen“ in der Datei vorkommt.
5. Nutzen sie `grep` um einen Satz zu finden der „studiert“ enthält.

Lösung

1. Die Terminal History kann nicht den gesamten Text der Datei anzeigen. Es ist daher nicht möglich den Anfang von „Faust“ zu lesen.
2. `more faust.txt` und `less faust.txt`. Hier geht es primär darum, eine Möglichkeit zu haben, den Inhalt sinnvoll im Terminal zu betrachten. `less` ist im allgemeinen besser.
3. `grep -c "Mephisto" faust.txt`—„Mephisto“ kommt 30 mal vor.
4. `grep -c "Gretchen" faust.txt`—„Gretchen“ kommt 21 mal vor.
5. `grep "studiert" faust.txt`

2.7.2 `grep` und Pipes

Das Kommando `grep` ist besonders nützlich wenn ein anderes Kommande eine große und unübersichtliche Ausgabe erzeugt. Wir erzwingen eine solche Situation indem wir mit unserem Package Manager nach „lib“ suchen. Dadurch werden alle möglichen Bibliotheken (von eng. „library“) oder „libre“-Software (von eng. „liberty“) gelistet werden. Diese Suche würden wir mit `dnf search lib` auf *Fedora* (bzw. `apt search lib` auf *Ubuntu*) auslösen. Filtern Sie die Ausgabe mithilfe von `grep` um nur die Ergebnisse zu erhalten, die sich auf das Audioformat „Vorbis“ beziehen (groß und kleinschreibung soll dabei keine Rolle spielen).

Lösung

`dnf search lib | grep -i "vorbis"` bzw. `apt search lib | grep -i "vorbis"`